SIMATS ENGINEERING

CSA02 - C Programming

ASSIGNMENT 1

CO1 | 50 marks

Reliable Billing for a Rural Electricity Co-operative

Design, implementation and validation

Complete solutions for Tasks 1-12

Student Name: PROBIR SAHA SHOHOM

Register Number: 1911260319

GitHub Repository:
Probir127/CSA02-1911260319-Assignment1

Student and scope

Student: PROBIR SAHA SHOHOM. Register Number: 1911260319. Course: CSA02 - C Programming. Programme: B.Tech - Computer Science and Engineering. Total marks: 50. Repository: https://github.com/Probir127/CSA02-1911260319-Assignment1.

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative. This report answers Tasks 1-12 with C source, the two completed-code exercises, pseudocode, a structure diagram, actual compiler output, test results and complexity comparisons. Generated input records are illustrative. Original tariff rules and smart-meter assumptions are stated explicitly rather than silently inferred.

Compiler: Apple Clang 21.0.0 invoked through gcc, C17, arm64 macOS. Every successful program was compiled and run; intentional compile/link/runtime failures are labelled as tests. Complete commands and expected return codes are in execution_commands.json. The supplied output images are renderings of captured output; terminal screenshots required for personal execution can be captured when running the documented commands. Faculty sharing and course submission remain separate steps.

Task 1 - Understand the problem and identify the data (4 marks)

Data, assumptions and invariants

The co-operative needs one monthly bill per household: energy charges for consumed kWh plus a fixed Rs 50 charge. This solution uses the assignment's worked tariff: the first 100 whole kWh are free, the next 100 cost 225 paise/kWh, and units above 200 cost 450 paise/kWh. Only domestic category D is supported because no other category tariff is specified. Unknown categories are rejected instead of borrowing an invented tariff.

Item	Representation and reason
Household number	Digit string up to 31 characters, preserving leading zeroes; an identifier, not an amount
Category	char D after checking the complete category token
Previous/current reading	Unsigned 16-bit register values in 0..65535; negative/fractional input is invalid
Confirmed rollover	Boolean Y/N field, supplied by a verified meter event or operator
Consumed units	unsigned whole kWh, at most 65535 under the period assumption
Energy, fixed and total charge	Non-negative integer paise; products and totals use long long
Batch total	Checked long long accumulator, potentially much larger than an individual bill
Tariff and slab boundaries	One shared set of constants for the original design; stable during a run
Rejected records	Line number plus reason, allowing correction without treating rejection as a zero bill

At most one rollover is allowed, with consumption strictly below 65536 units per period. If current < previous, the Y flag is required. If current >= previous, Y is inconsistent with this assumption and is rejected. Equal readings therefore mean zero consumption; a complete 65536-unit cycle cannot be inferred from readings alone. The flag is necessary because a falling reading could be a transcription error, a replacement meter, or a rollover. The arithmetic cannot establish the cause.

A correct bill has non-negative units and charges; total equals energy plus fixed; increasing units cannot reduce the bill under the supplied rates; and accumulated totals must not overflow. The original input batch is assumed to contain one record per household for one billing month. Duplicate household/month detection, replacement-meter workflows, tax,

subsidy, arrears and payment handling are outside this exercise and need additional records/rules before real deployment.

Text readings as an alternative

A reading such as 01530 can be stored in char[6] including the terminating zero, while uint16_t uses two bytes here. Validated conversion requires O(d) work for d digits and checks for signs, non-digits and out-of-range values. Comparing arbitrary-length numeric strings requires removing leading zeroes, comparing significant lengths, then digits: O(d), not ordinary lexicographic comparison (which orders 100 before 20). Integer comparison is O(1) for these fixed-width types. The final input implementation deliberately reads bounded text and converts once, then keeps numeric readings for calculation.

Program result

```
Household=00001234 category=D previous=1200 current=1530 units=330
Energy: Rs 810.00
Fixed: Rs 50.00
Total: Rs 860.00
Nonnegative and sum invariant: PASS
```

Task 2 - Choose the data types (4 marks)

Measured sizes and ranges

The command named gcc on this Mac is Apple Clang 21.0.0, targeting arm64-apple-darwin27.0.0. The examples were compiled as C17. They use the requested gcc command syntax; these are not claimed as GNU GCC results. sizeof reports C bytes; CHAR_BIT reports the number of bits in each byte.

```
CHAR_BIT=8
char: 1 bytes, -128..127
signed char: 1 bytes, -128..127
unsigned char: 1 bytes, 0..255
short: 2 bytes, -32768..32767
unsigned short: 2 bytes, 0..65535
int: 4 bytes, -2147483648..2147483647
unsigned int: 4 bytes, 0..4294967295
long: 8 bytes, -9223372036854775808..9223372036854775807
unsigned long: 8 bytes, 0..18446744073709551615
long long: 8 bytes, -9223372036854775808..9223372036854775807
unsigned long long: 8 bytes, 0..18446744073709551615
float: 4 bytes, normal min=1.17549435e-38 max=3.40282347e+38 digits=6 mantissa=24
double: 8 bytes, normal min=2.2250738585072014e-308 max=1.7976931348623157e+308 digits=15
    mantissa=53
long double: 8 bytes, max=1.79769313e+308 mantissa=53
16-bit register after 65535 + 1: 0
```

The chosen core types are uint16_t for the hardware register, unsigned for consumed units, char arrays for identifiers, char for category, bool for confirmation, and long long for paise and batch totals. uint16_t is available on this platform and its exact 16-bit range is asserted. The exercise's unsigned short is also explicitly asserted to have maximum 65535. A 16-bit register increment wraps to zero through conversion to the unsigned type; this does not imply signed overflow is allowed.

C guarantees at least 8 bits in a byte, short/int ranges of at least -32767..32767, long at least -2147483647..2147483647 and long long at least -9223372036854775807..9223372036854775807. Unsigned counterparts cover at least 0..65535, 0..4294967295 and 0..18446744073709551615 respectively for short/int, long and long long. sizeof(char) is always 1; plain char signedness is implementation-defined. This machine has 8-byte long, but a platform with 4-byte long is conforming. Never assume long is always district-safe.

float has a large exponent range, but only 24 binary precision bits here; double has 53. FLT_MIN/DBL_MIN above are the smallest positive normal values, not the most negative values or the smallest subnormal values. The standard minimum decimal precision is 6 digits for float and 10 for double; this machine provides 6 and 15 through FLT_DIG/DBL_DIG.

Exact money and a measured float failure

A maximum original-period bill is 29,428,250 paise = Rs 294,282.50. Adding that valid bill 4,000 times should give Rs 1,177,130,000.00. The float accumulator gives Rs 1,177,099,648.00: an understatement of Rs 30,352.00. The integer result is exactly 117,713,000,000 paise. This is a deliberately high-consumption stress case, not a statement about typical household usage.

The original tariff's quarter-rupee increments are binary-representable; the demonstrated error comes from precision lost as the accumulated total becomes large, not from pretending that 2.25 itself is inexact. Future decimal tariffs can introduce representation error even before accumulation. Integer paise preserves amounts exactly until overflow, which must still be checked.

Completed rollover code and explanation

Only the TODO region was filled; it adds a compile-time width assertion and returns (unsigned short)(current - previous), without if-else. On this system the operands promote to int, subtraction gives -65280, and conversion to 16-bit unsigned reduces modulo 65536. Thus the rollover result is 136 remaining increments to zero plus 120 further increments = 256.

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

Alternative: double rupees

A double design can compute a bill in rupees, validate finite/range bounds, and round half-up to paise at each bill boundary before adding final monetary amounts. Rounding only the final district sum can differ from summing individually issued bills. Both fixed-width additions are O(1); double and long are both 8 bytes on this machine, while long can be 4 bytes elsewhere. double exactly represents the quarter-rupee values in this particular tested range, so it matched these totals, but that does not prove arbitrary decimal tariff computations are exact. Binary rounding error depends on values, ordering and precision; it is not necessarily monotonic or always in one direction.

Task 3 - Define the constants (3 marks)

Central tariff and completed exercise

The production examples share common/billing.h. SLAB1_UNITS, SLAB2_UNITS, RATE1_PAISE, RATE2_PAISE, RATE3_PAISE and FIXED_CHARGE_PAISE have uppercase names that expose units and purpose. Wide money literals use LL, and operands are widened before multiplication. The separate supplied Complete the Code exercise retains its long return type and original macros; its TODO implements the three marginal blocks from highest to lowest. Negative units are outside that helper's valid domain and are rejected by the input layer.

```
0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Changing only RATE2_PAISE from 225 to 230 in revised_rate.c produces:

```
0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

The 0- and 100-unit bills correctly remain Rs 50.00 because their second-slab consumption is zero. The 150-unit bill rises by 50 x 5 = 250 paise; the 200- and 350-unit bills each rise by 100 x 5 = 500 paise. A consistent tariff revision changes every affected bill, not necessarily every bill.

#define, const and enum

#define performs token replacement before C type checking, has no run-time object/storage, and is also useful in #if expressions. Parenthesise expression-like replacements and choose a suitable suffix. A typed const variable has C scope and type checking and can have an address/debugger representation; the optimizer may remove its storage. In C17 a const int is not generally an integer constant expression for a case label or file-scope array bound. enum is suitable for small integral named choices such as categories/status codes; C17 enumerator values must fit int, so it is not the general solution for very large monetary limits.

Replacing macros with static const money_t rates and const unsigned boundaries preserves O(1) work per fixed-three-slab bill. A compiler can fold either form. Memory differs only if const objects are retained; #define and enum constants create no corresponding addressable run-time object. A tariff revision is made in the one shared header, then all dependent translation units are rebuilt together. Tasks 11-12 show why runtime tariff data is preferable when revisions must not require a rebuild.

Task 4 - Use macros safely (4 marks)

Expansion and pitfalls

The examples use SQUARE(x) ((x)*(x)), MAX(a,b) (((a)>(b))?(a):(b)) and RUPEES_TO_PAISE(r) ((r)*100LL). The last macro is for whole non-negative rupees within the multiplication range, not arbitrary fractional rupees. Full parentheses fix grouping; they do not validate range or remove duplicated evaluation.

```
BAD_SQUARE(a+1)=7; SQUARE(a+1)=16
MAX(i++,j)=6; i after=7
max_int(i++,j)=5; i after=6
square_int(a+1)=16; 50 rupees=5000 paise
```

For a=3, BAD_SQUARE(a+1) expands to a+1*a+1, which is 7 rather than 16. Parentheses give ((a+1)*(a+1)). For i=5,j=3, MAX(i++,j) increments i during the condition, then evaluates the selected i++ arm again: result 6 and final i 7. These two evaluations are sequenced by the conditional operator. By contrast, SQUARE(i++) has unsequenced modifications and must not be used; no undefined-behaviour output is offered as a portable result.

The actual gcc -E -P output is retained in Task04/preprocessed.i. Its main function contains the expanded expression tokens; includes and macro definitions have already been processed. static inline square_int and max_int evaluate each argument once. The function call example gives result 5 and final i 6.

Alternative: static inline functions

Use functions for nontrivial calculations, validation, side-effectful inputs and clearer type contracts. Parameter types constrain conversions. Both forms are O(1) per small operation. A non-inlined function has call/return overhead; static inline permits but does not force inlining. With k uses, macro-expanded source grows O(k); emitted machine code depends on optimization and may merge/fold expressions. An outlined function has one body plus O(k) call sites, while an inlined body can also grow O(k). It is incorrect to claim that macro expansion necessarily runs faster or always creates a larger final binary.

Task 5 - Headers and conditional compilation (3 marks)

Structure and header guards

common/billing.h holds public types, constants and prototypes. common/billing.c includes it and implements consumption, tariff arithmetic, money formatting and checked addition. Each task's main function coordinates the demonstration. Task09 further separates main.c and input.c. Global declarations define interfaces and constants; mutable billing state is local to a call or account.

The header uses #ifndef CSA02_BILLING_H / #define CSA02_BILLING_H / #endif. Task05 intentionally includes it twice, demonstrating that typedef/struct declarations are not duplicated. #include <stdio.h> searches the implementation's configured system include paths; #include "../common/billing.h" first uses the implementation's

quoted-header search rules, normally relative to the including source, then configured paths. Exact search order is implementation-specific, so the distinction is not a universal absolute-path rule.

The DEBUG build includes a diagnostic fprintf inside bill_for_units; the normal build removes that block during preprocessing. Its diagnostic is written to stderr.

```
Header guards work; total=Rs 162.50

Header guards work; total=Rs 162.50
DEBUG units=150 energy=11250 fixed=5000 total=16250
```

Alternative: runtime debug flag

A runtime design uses if (debug_enabled) fprintf(...). In a loop over n bills, an actually retained branch is O(n) tests, each O(1), and debug instructions/strings remain in the executable. If the flag is compile-time-known, optimization may eliminate the branch. #ifdef removes the entire block in a release build, so there is no debug test or debug output code from that block. Enabled output has cost proportional to message bytes and I/O overhead; this can dominate either branch strategy. A runtime flag is more convenient for diagnosing a deployed executable; a compile-time flag gives a simpler release binary.

Task 6 - Read and print data correctly (4 marks)

Format specifiers and safe conversion

Data type	scanf destination	printf argument
int	%d with int*	%d
unsigned int	%u with unsigned int*	%u
long	%ld with long*	%ld
long long	%lld with long long*	%lld
float	%f with float*	%f: float promotes to double
double	%lf with double*	%f (printf %lf has the same floating conversion in C17)
char	%c with char*; leading space in format skips whitespace	%c: promoted to int
char array	width-limited %31s into char[32]	%s with a terminated string
uint16_t / uint64_t	Exact-width SCN macros or checked text conversion	PRI macros or a validated cast to a matching type

scanf_demo checks that two assignments succeeded, then validates 0..65535. The saved valid, letter and negative input cases show both parsing and range rejection:

```
Enter previous and current readings: scanf assignments=2
Accepted previous=1200 current=1530
Example formatted bill: Rs 162.50

Enter previous and current readings: scanf assignments=0
Invalid numeric input

Enter previous and current readings: scanf assignments=2
Reading outside 0..65535
```

Integer paise is printed using quotient and a two-digit remainder: p/100 and p%100. This is exact for a non-negative validated monetary amount and prints Rs 162.50 for 16250 paise.

Wrong-specifier test

Task06/wrong_format.c deliberately passes a long to printf %d. Compiling with -Werror=format rejects it; the diagnostic is saved in format_diagnostic.txt. A mismatched variadic format has undefined behaviour, so there is no portable meaningful runtime answer to promise. A scanf %d destination pointing to long is also wrong and can overwrite only part of an object or worse. The corrected form is printf("%ld", amount) and scanf("%ld", &amount), or the appropriate fixed-width macro. The deliberately invalid example is not executed.

Alternative: fgets plus checked parsing

A direct numeric scanf conversion also cannot safely handle an arbitrary numeric token outside the destination type's representable range merely by checking its return value. It can leave an invalid token in the stream, making repeated reads fail. The robust Task09 program therefore reads each bounded line with fgets, detects/discards an overlong line, parses bounded string tokens with sscanf and converts digit-only readings through strtoul with errno/end-pointer/range checks. Negative strings are rejected before conversion.

Both approaches inspect O(L) characters for a line of length L. fgets requires an O(L) buffer if the whole line is retained, or a fixed-cap buffer plus explicit overlength rejection as used here. Direct scanf can avoid a full line buffer, but recovery requires consuming the rest of the failed line; %s still needs a bounded destination. Line-based parsing makes each rejection independent of the next input record.

Task 7 - Type conversion and precision (4 marks)

Demonstrated surprises

```
150/100=1; (double)150/100=1.50
-1 < 1U = 0 (after unsigned conversion)
Signed-domain comparison corrected = 1
unsigned char 255 + 1 = 256 (integer promotion)
(int)(INT_MAX+1L)=-2147483648 (implementation-defined in C17)
float 16777216 + 1 = 16777216
llround(1.125*100)=113 paise (half away from zero)
```

150/100 performs integer division before any later assignment; casting an operand before division produces 1.50. The signed -1 becomes a large unsigned value in comparison with unsigned 1, so the original comparison is false. The corrected signed-domain comparison first handles negativity, then casts only a non-negative value. The compiler's -Wsign-compare warning for the original teaching expression is retained in warning.txt; billing comparisons use compatible types.

unsigned char promotes to int on this machine, so 255+1 is the int value 256 rather than an 8-bit wrap. Narrowing a large signed long to int when it is outside range is implementation-defined in C17; the observed -2147483648 is not a cross-platform contract. At 2^24, this machine's float spacing becomes 2, so adding one can leave the value unchanged.

Conversion points and rounding rule

Text readings become unsigned long only after digit validation, then narrow to meter_t after checking 65535. Rollover subtraction is deliberately converted modulo 2^16. Unit-rate multiplication widens the unit count to money_t before multiplying, avoiding a narrow intermediate. Printf arguments either match their format exactly or are explicitly cast to the declared matching type. District additions check LLONG_MAX before adding.

The original whole-kWh tariff needs no rounding because every rate is an integer number of paise. For an externally calculated non-negative fractional charge, the chosen rule is nearest paisa with exact half rounded upward. llround demonstrates half-away-from-zero on a positive exact example; validate finiteness and destination range before calling it. Multiplying a binary double by 100 can be slightly off a decimal halfway value, so it is not a universal exact decimal implementation. Task12 instead retains an integer rational numerator and performs half-up rounding once at the final energy-charge boundary.

Alternative: float everywhere and one final rounding

Task07/precision.c runs the same 4,000/60,000/800,000-bill stress experiment as Task10, with three measured repetitions retained in precision_runs.json. Float's accumulating error changes sign and grows substantially in these examples; rounding once at the end cannot recover bits already lost. An integer addition and a float addition are both O(1); float uses 4 bytes and money_t 8 bytes here. Floating-point error depends on magnitudes and ordering: a standard repeated-addition bound has a factor approximately n*u for small n*u times the sum of absolute terms, not a guaranteed fixed error per bill. Exact integer arithmetic has zero rounding error within range, but overflow must be prevented.

Task 8 - Trace compilation (4 marks)

Four stages and observed artifacts

The following commands were executed; build holds local object/executable artifacts, while the text outputs are included with this task.

```
gcc -E Task08/stages.c -o Task08/stages.i
gcc -std=c17 -Wall -Wextra -S Task08/stages.c -o Task08/stages.s
gcc -std=c17 -Wall -Wextra -c Task08/stages.c -o build/stages.o
gcc build/stages.o -o build/stages
./build/stages
```

Preprocessing expands the CHARGE macro to 5000L, includes header declarations, handles conditional compilation and removes comments. The .i file contains resulting C tokens and source-location directives, not executable instructions. Compilation produces target assembly in .s. Assembly creates a relocatable object containing machine code and symbol/relocation information. Linking resolves references such as printf against the runtime/libraries and creates the executable. The machine here produces arm64 Mach-O artifacts; a GNU/Linux system would normally produce ELF, so binaries are rebuilt rather than distributed as portable deliverables.

```
{
  "stages.c": 111,
  "stages.i": 25825,
  "stages.s": 1035,
  "stages.o": 760,
  "stages": 33432
}

Fixed charge=5000 paise
```

Errors and warnings, correctly classified

compile_error.c references undeclared missing_variable; translation fails before an object can be linked. link_error.c has a valid declaration of missing_tariff but no definition; compilation can succeed, yet linking reports an undefined symbol. runtime_error.c builds successfully, then tries to open a deliberately absent input file, reports the failure and exits 2. These are respectively a compilation error, a link error and a handled runtime failure; they are separate from undefined behaviour.

-Wall -Wextra identified the wrong printf format and the mixed-sign comparison. The production code uses the correct format, explicit validated conversions and compatible types. The original mixed-sign expression is retained only in the teaching demonstration with its warning and a corrected expression alongside it. Additional AddressSanitizer/UndefinedBehaviorSanitizer test builds completed without reported faults in the tested billing and smart-meter paths.

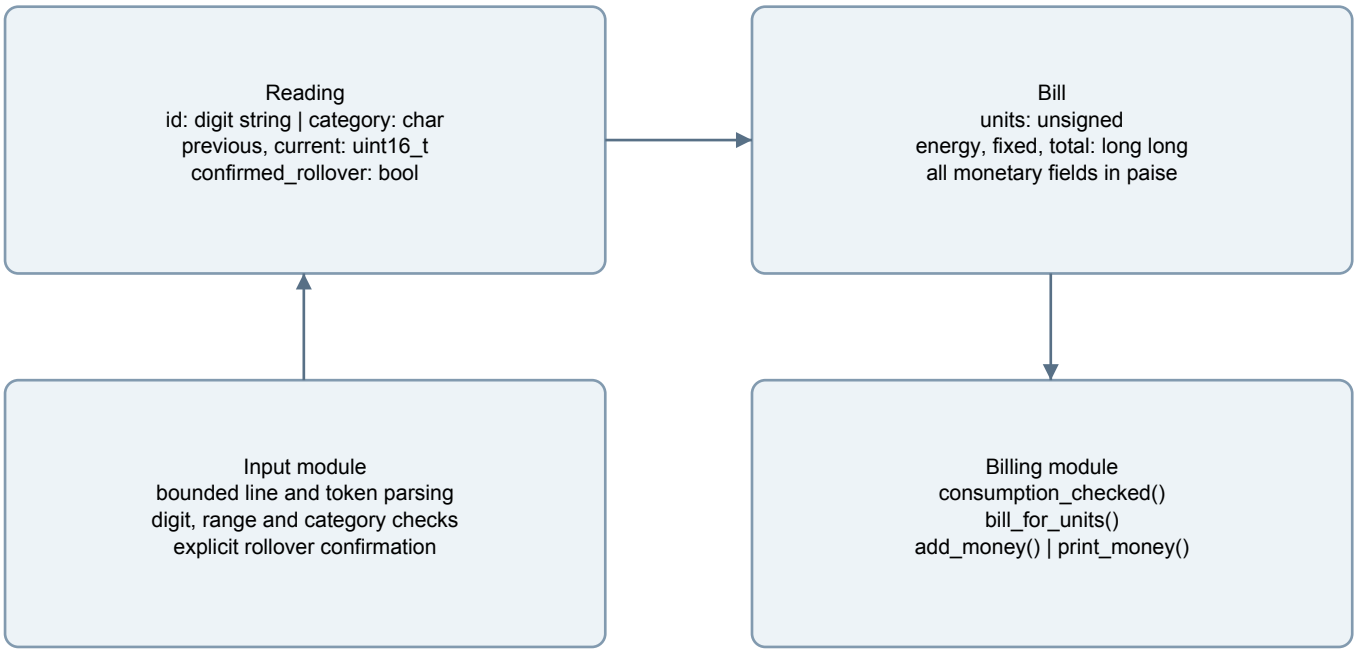
Alternative: separate translation units

The final billing executable was also built from separate objects for Task09/main.c, Task09/input.c and common/billing.c, then linked; its five-bill output exactly matches the single compile command. A change only to input.c needs recompiling that translation unit and relinking, whereas a monolithic source needs recompiling its whole body. A shared-header change can require every dependent file to rebuild; a dependency-aware build system must track that. Compile time roughly follows the amount of translated source and optimization work, not simply the number of files. Multiple object files consume the combined machine-code/data/symbol storage plus per-file overhead; the linked result can still have similar executable size.

Task 9 - Implement and test the billing program (4 marks)

Architecture and pseudocode

The caller supplies a line containing household category previous current rollover, for example 0003 D 0 150 N. The main program owns input iteration and the district total; input.c validates the source fields; common/billing.c computes units and money. Invalid lines are reported and skipped, while accepted lines continue. Exit 0 means all supplied records were accepted; exit 2 means at least one was rejected; exit 3 reports an I/O or aggregate-overflow failure.




```
ALGORITHM GENERATE_MONTHLY_BILLS
INPUT: bounded lines of household/category/readings/rollover
OUTPUT: accepted bills, reject reasons, checked batch total
BEGIN
    SET batch total TO zero
    FOR EACH input line DO
        IF line is too long or fields are invalid THEN
            REPORT rejection and CONTINUE
        ENDIF
        IF the decrease and rollover confirmation disagree THEN
            REPORT rejection and CONTINUE
        ENDIF
        COMPUTE units using validated 16-bit modular difference
        COMPUTE marginal energy charge with wide integer products
        ADD fixed charge TO form the bill total
        IF adding the bill would exceed the accumulator range THEN
            REPORT total overflow and STOP
        ENDIF
        ADD bill TO batch total
        OUTPUT household, units and bill components
    ENDFOR
    OUTPUT accepted count, rejected count and batch total
END
```

Expected versus actual results

The expected values below were independently hand-calculated at the slab boundaries; the C assertion suite compares them to actual results.

```
units=0 expected=5000 actual=5000 PASS
units=99 expected=5000 actual=5000 PASS
units=100 expected=5000 actual=5000 PASS
units=101 expected=5225 actual=5225 PASS
units=150 expected=16250 actual=16250 PASS
units=199 expected=27275 actual=27275 PASS
units=200 expected=27500 actual=27500 PASS
units=201 expected=27950 actual=27950 PASS
units=350 expected=95000 actual=95000 PASS
units=65535 expected=29428250 actual=29428250 PASS
65,536 array-vs-fixed comparisons; rollover, parser, sum and overflow guards: PASS
Five hand-calculated bills total: 148750 paise = Rs 1487.50
```

The five hand bills are Rs 50.00 + Rs 50.00 + Rs 162.50 + Rs 275.00 + Rs 950.00 = Rs 1487.50. Their execution is:

```
0001 units=0 energy=Rs 0.00 fixed=Rs 50.00 total=Rs 50.00
0002 units=100 energy=Rs 0.00 fixed=Rs 50.00 total=Rs 50.00
0003 units=150 energy=Rs 112.50 fixed=Rs 50.00 total=Rs 162.50
0004 units=200 energy=Rs 225.00 fixed=Rs 50.00 total=Rs 275.00
0005 units=350 energy=Rs 900.00 fixed=Rs 50.00 total=Rs 950.00
Accepted=5 rejected=0 total=Rs 1487.50
```

The separate invalid/rollover file demonstrates letter input, negative input, a decreasing reading without confirmation, both rollover examples, a reading above 65535 and an inconsistent Y flag:

```
REJECT line 1: invalid fields/range/category
REJECT line 2: invalid fields/range/category
REJECT line 3: inconsistent or unconfirmed rollover
0009 units=1 energy=Rs 0.00 fixed=Rs 50.00 total=Rs 50.00
0010 units=256 energy=Rs 477.00 fixed=Rs 50.00 total=Rs 527.00
REJECT line 6: invalid fields/range/category
REJECT line 7: inconsistent or unconfirmed rollover
Accepted=2 rejected=5 total=Rs 577.00
```

Additional parser assertions reject unknown category, trailing data and an overlarge numeric token. The overflow test attempts to add one to LLONG_MAX and checks that the accumulator remains unchanged. Sanitized builds run the same valid invariants and invalid-input paths. An extra bounded-line test in the execution harness confirms that a long malformed line is rejected without corrupting the following valid line.

Alternative: arrays of slabs

The test suite also implements widths [100,100,unbounded] and rates [0,225,450], iterating across the remaining units. It agrees with the fixed code for all 65,536 possible unit values. A loop across s slabs costs O(s) time and O(s) tariff storage; the original three fixed tests are O(1) because their count never changes. Generalising a hard-coded chain to s tests also makes it O(s). Arrays are easier to extend or load from configuration, but boundaries, lengths, ordering and arithmetic limits must be validated once before use.

Task 10 - Expectations versus execution (4 marks)

Measured totals and overflow checks

Every test bill uses 65535 units, the largest admissible original-period consumption, so exact totals can be independently checked by multiplying 29,428,250 paise by n. The experiment uses volatile accumulators to ensure each addition is executed, without fast-math reassociation.

Bills	Exact paise	Float error (paise)	Double error (paise)	int / long / long long safe
4000	117713000000	-3035200	0	0 / 1 / 1
60000	1765695000000	-802328000	0	0 / 1 / 1

Bills	Exact paise	Float error (paise)	Double error (paise)	int / long / long long safe
800000	23542600000000	45780672000	0	0 / 1 / 1

Here 1 means the entire total is representable, and 0 means it is not. The exact district total is 23,542,600,000,000 paise, well within this machine's 64-bit long and long long, but outside 32-bit int. The first maximum-bill addition that would exceed INT_MAX is number 73. The program tests representability before addition rather than executing signed overflow, whose behaviour is undefined. A 32-bit long platform also fails this district requirement, despite being valid C.

The float difference is -3,035,200 paise at 4,000 bills, -802,328,000 paise at 60,000, and +45,780,672,000 paise at 800,000. The different signs show why a universal fixed correction is invalid. Integer paise matches the independently computed product exactly, and double rupees also matches for this quarter-rupee dataset and magnitude.

Timing and storage

Bills	Float ms	Integer ms	Double ms
4000	0.019000	0.008000	0.015000
60000	0.230000	0.120000	0.233000
800000	3.189000	0.414000	2.963000

These are CPU-clock medians from three local runs, in milliseconds, with raw runs retained. Very small differences are sensitive to clock granularity, optimization and background work. The volatile loop deliberately prevents elision, so the results demonstrate the implemented additions rather than a universal performance ranking. Total runtime is O(n), each fixed-width addition O(1), and the three accumulators require O(1) extra memory. The integer and double variables are 8 bytes each; float is 4 bytes here.

Alternative: a double district total

Double provides enough precision for this measured 800,000-bill case. As an integer-paise accumulator, a binary64 double can represent every integer only through 2^53; above that some adjacent paise cannot be distinguished even though the exponent range is far from exhausted. As rupees, its unit spacing must remain small enough for the required decimal cents, and decimal rates can introduce earlier rounding error. A long long retains every integer paise within its range, then overflows if unchecked. Both use 8 bytes and O(1) additions on this machine; exact fixed-point money with explicit overflow guards offers the clearest invariant.

Task 11 - Recommend and defend the design (4 marks)

Recommendation supported by results

Use uint16_t only for the old hardware register; use unsigned units after validated conversion; keep household numbers as bounded strings; use char for validated category and bool for rollover confirmation; use long long paise for individual charges and aggregate money. Define tariff boundaries and rates once in an include-guarded header, with wide multiplication operands. Prefer functions over side-effect-sensitive calculation macros, keep a controlled DEBUG build, and compile with warnings enabled.

The evidence is concrete: the two rollover examples match 256 and 1 units; all 65,536 tariff calculations agree with an independent slab-array implementation; the five bills total 148750 paise; a 32-bit int is inadequate but long long holds the 23,542,600,000,000-paise district bound; float produces substantial errors. The tested ordinary and smart-meter paths also pass sanitizer checks.

Change the original tariff in the shared definition, rebuild all dependants, rerun the boundary/revision tests, and deploy one consistent version. The design remains arithmetically correct at district scale under its stated maximum consumption and tariff. It does not automatically solve ambiguous rollover, duplicate monthly records, meter

replacement, authentication, persistence or concurrent district processing. Those require additional business and storage controls.

```
Maximum monthly bill=29428250 paise
District bound=23542600000000 paise
long long maximum=9223372036854775807; bound fits=yes
Use checked integer paise, a confirmed rollover flag, and one tariff definition.
```

Alternative: startup configuration

Read t tariff values from a versioned file, validate required names, duplicates, numeric syntax, bounds and slab ordering, then keep a typed tariff object fixed for the run. Parsing is O(total configuration bytes), commonly O(t) for bounded-length values; retained storage is O(t). This small startup cost avoids recompiling and redistributing binaries for rate-only changes. Runtime configuration must not change midway through an account's bill, and a real bill should retain the tariff version used. Task12 implements this approach and proves that changing one rate changes the same executable's result.

Task 12 - Re-evaluate and adapt the design (8 marks)

A. Impact analysis (2 marks)

Retain integer money, validation, bounded input, functions, header guards and warning/test practices. Modify the old 16-bit/monthly-reading model to wider cumulative watt-hour readings and timestamped 15-minute deltas. Redesign tariff storage as validated runtime data and separate exact fractional charge accumulation from final paisa rounding.

There are 96 intervals/day and 35,040 intervals/365-day year per household. Annual network record counts are 140,160,000 for 4,000 homes; 2,102,400,000 for 60,000; and 28,032,000,000 for 800,000. The district record count alone exceeds a 32-bit unsigned counter. A leap year has more intervals; the given 35,040 figure assumes 365 days.

The brief gives no maximum power draw or meter lifetime capacity, so a numeric maximum cannot be inferred from interval frequency alone. The implementation therefore declares and enforces a configurable engineering cap of at most 1,000,000 Wh per interval, not a claim about a real household's consumption. Rates are capped at 100,000 paise/kWh, surcharge at 100%, fixed charge at 1,000,000 paise/month, and the second slab boundary at 1,000,000,000 Wh. These conservative bounds make the range analysis explicit and are adjustable only through a reviewed design change if exceeded.

B. Evaluate the original design with bounds (2 marks)

Quantity	Bound under the declared limits	Consequence
One-house annual energy	35,040 x 1,000,000 = 35,040,000,000 Wh	Too large for 16-bit or 32-bit unsigned
District annual energy	28,032,000,000,000,000 Wh	Fits uint64_t; far beyond 32-bit
One-house annual numerator	35,040,000,000 x 100,000 x 200 = 700,800,000,000,000,000	Fits uint64_t; check multiply/add before computing
One-house annual paise upper bound	7,008,000,000,000 + 12,000,000 fixed = 7,008,012,000,000	Fits 64-bit integer money
District annual paise upper bound	5,606,409,600,000,000,000	Below signed 64-bit maximum 9,223,372,036,854,775,807

The numerator is an exact rational charge scaled by 100000, not an amount already in paise. Summing every household's unrounded numerator could exceed 64 bits; finish each household/month in paise first, then use checked aggregation of the issued bills. Do not rely on a 64-bit type alone without operation-specific overflow checks. uint64_t cumulative device readings can still exhaust their lifetime range; this revised demonstration rejects decreases rather than assuming a new meter's rollover modulus.

Rates, fixed charges, slab thresholds and peak-window parameters must now be runtime tariff fields, not #define values requiring a rebuild. Technical constants such as the 15-minute slot model, scale denominator and input buffer size can remain compile-time constants.

C. Implement the revised design (2 marks)

Task12 loads ten key=value fields from tariff.cfg. It rejects unknown/missing/duplicate keys, negative or malformed numbers, overlarge rates, invalid boundaries and peak windows. A Tariff struct holds the validated values. One Account struct holds cumulative period Wh, a rational charge numerator and interval count. Arithmetic helpers reject overflow; apply_interval updates a temporary account and commits it only after every operation succeeds.

The original marginal monthly slabs are retained, now expressed in Wh: 100000 and 200000. Consumption is allocated to slabs in timestamp order, including intervals that cross a threshold. The current interval's base marginal energy charge receives a 20% uplift if its start time is in [18:00,22:00). The fixed charge is not multiplied by the peak factor. This is an explicit interpretation of the changed tariff, since the brief does not specify whether the slabs disappear or how crossing intervals should be allocated. Replacing this interpretation requires revising the business rule and its tests.

For each portion, exact numerator += Wh x rate_paise_per_kWh x percentage_factor; dividing by 1000 x 100 converts Wh and percentage to paise. Retain the numerator through the period and round once: quotient plus one when the remainder is at least 50000. This handles 0.45-paisa subcharges without rounding each to zero.

The input command processes one household's supplied billing-period sequence: day number (1..31), slot (0..95), previous cumulative Wh and current cumulative Wh. Consecutive records must have consecutive slots and matching boundary readings. It rejects missing, duplicate, decreasing, over-limit or discontinuous intervals. The example permits a partial sequence for demonstration and prints a coverage reminder; the caller must establish the actual billing calendar/month and complete coverage before issuing a production bill. Each new billing month needs a fresh account so the free slab resets and the fixed charge is applied once. Times are interpreted as local slot starts; timezone/DST and clock-correction policies need explicit production metadata.

```
ALGORITHM BILL_SMART_PERIOD
INPUT: validated tariff and ordered 15-minute readings for one account/month
OUTPUT: exact rounded energy charge and fixed-plus-energy total
BEGIN
    LOAD and VALIDATE tariff once
    SET consumed Wh, charge numerator and interval count TO zero
    FOR EACH contiguous interval DO
        CHECK readings, timestamp order and interval limit
        SET remaining Wh TO current reading minus previous reading
        SET percentage factor TO 120 in peak slots, otherwise 100
        WHILE remaining Wh is positive DO
            SELECT marginal slab using period consumption
            TAKE Wh up to the next slab boundary
            CHECK all multiplications and additions for overflow
            ADD Wh x slab rate x percentage factor TO numerator
            ADVANCE consumed Wh and remaining Wh
        ENDWHILE
    ENDFOR
    ROUND numerator / 100000 once, half upward
    ADD one fixed charge with an overflow check
    RETURN energy and total; require complete-period coverage for issuance
END
```

D. Evidence-based defence (2 marks)

```
Intervals=4 used_Wh=350000
Energy=108000 paise; fixed=5000 paise; total=Rs 1130.00
One supplied billing-period sequence; coverage must be checked before issuing a bill.

Intervals=4 used_Wh=350000
Energy=108600 paise; fixed=5000 paise; total=Rs 1136.00
One supplied billing-period sequence; coverage must be checked before issuing a bill.

350 kWh mixed/slab-crossing test: energy=108000 total=113000 paise PASS
slot=71 (start 17:45) energy=450 paise PASS
slot=72 (start 18:00) energy=540 paise PASS
slot=87 (start 21:45) energy=540 paise PASS
slot=88 (start 22:00) energy=450 paise PASS
Two 0.45-paisa intervals -> 1 paisa after one final rounding: PASS
Overflow and over-limit rejection leave account unchanged: PASS
```

```
Annual interval traversal=35040 count; total=35040000 Wh PASS
```

The first 100 kWh are free. The next 100 kWh at peak cost 100 x 225 x 1.20 = 27000 paise; the next 150 cost 150 x 450 x 1.20 = 81000 paise. Energy is 108000, fixed is 5000, and total is 113000 paise. Changing rate2 to 230 in the configuration raises the total to 113600 without recompilation. Slots 71/72/87/88 verify 17:45, 18:00, 21:45 and 22:00 boundary behaviour. Two off-peak 1-Wh charges at 450 paise/kWh produce 0.9 paisa together and correctly round to 1 paisa at the end.

The tests traverse 35,040 intervals and verify total Wh; this checks annual iteration/count range, not a claim to issue a yearly bill with only one monthly slab or fixed charge. Separate negative tests reject repeated/invalid configuration, missing intervals, decreasing counters, an excessive delta and a duplicated slot. Overflow rejection leaves the account unchanged. A rational-numerator stress bound supports the choice of 64-bit state; full district event data is not materialised or falsely claimed as an executed 28-billion-row benchmark.

Each interval crosses at most three slabs, so the current fixed-three-slab computation is O(1) per interval and O(N) for N intervals. A configurable s-slab version is O(s) per crossing calculation in the simple implementation. Streaming one account uses O(1) calculation state plus its tariff; processing many interleaved households needs O(h) active-account state or persistent storage. Binary-only configuration updates, calendar validation, immutable tariff versions, source authentication and a database of issued bills are the next production concerns; the classroom implementation demonstrates the revised arithmetic and validation contract.

References

The supplied CSA02 CO1 assignment defines the scenarios and task marks. The GCC manual's Overall Options page documents the -E, -S and -c stopping stages: https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html. The GCC preprocessor manual explains duplicated side effects in function-like macros: https://gcc.gnu.org/onlinedocs/cpp/Duplication-of-Side-Effects.html. These references describe the concepts; the compiler actually executed here is the Apple Clang version recorded in compiler.txt.

Appendix - C source listings

Each listing is labelled with its folder and filename. Related files are included in the companion ZIP and repository. Deliberate error examples are identified in their task explanations. Long lines are visually wrapped; use the original source files for execution.

Task01/data_demo.c

```
#include <stdio.h>
#include "../common/billing.h"
int main(void) {
    const char household[]="00001234";
    const char category='D';
    meter_t previous=1200, current=1530;
    unsigned units=0;
    if (!consumption_checked(previous,current,false,&units)) return 1;
    Bill b=bill_for_units(units);
    printf("Household=%s category=%c previous=%u current=%u units=%u\n",
           household,category,(unsigned)previous,(unsigned)current,units);
    printf("Energy: ");print_money(b.energy_paise);
    printf("\nFixed: ");print_money(b.fixed_paise);
    printf("\nTotal: ");print_money(b.total_paise);putchar('\n');
    printf("Nonnegative and sum invariant: %s\n",
           b.total_paise>=0 && b.total_paise==b.energy_paise+b.fixed_paise?"PASS":"FAIL");
    return 0;
}
```

Task02/complete_code.c

```
#include <stdio.h>
#include <limits.h>
/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;
unsigned int consumption(meter_t previous, meter_t current)
{
    _Static_assert(USHRT_MAX == 65535, "This exercise requires 16-bit unsigned short");
    return (unsigned short)(current - previous);
}
int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));   /* rolled over */
    return 0;
}
```

Task02/types.c

```
#include <stdio.h>
#include <limits.h>
#include <float.h>
#include <stdint.h>
int main(void) {
    printf("CHAR_BIT=%d\n", CHAR_BIT);
    printf("char: %zu bytes, %d..%d\n",sizeof(char),CHAR_MIN,CHAR_MAX);
    printf("signed char: %zu bytes, %d..%d\n",sizeof(signed char),SCHAR_MIN,SCHAR_MAX);
    printf("unsigned char: %zu bytes, 0..%u\n",sizeof(unsigned char),(unsigned)UCHAR_MAX);
    printf("short: %zu bytes, %d..%d\n",sizeof(short),SHRT_MIN,SHRT_MAX);
    printf("unsigned short: %zu bytes, 0..%u\n",sizeof(unsigned short),(unsigned)USHRT_MAX);
    printf("int: %zu bytes, %d..%d\n",sizeof(int),INT_MIN,INT_MAX);
    printf("unsigned int: %zu bytes, 0..%u\n",sizeof(unsigned),UINT_MAX);
    printf("long: %zu bytes, %ld..%ld\n",sizeof(long),LONG_MIN,LONG_MAX);
    printf("unsigned long: %zu bytes, 0..%lu\n",sizeof(unsigned long),ULONG_MAX);
    printf("long long: %zu bytes, %lld..%lld\n",sizeof(long long),LLONG_MIN,LLONG_MAX);
    printf("unsigned long long: %zu bytes, 0..%llu\n",sizeof(unsigned long long),ULLONG_MAX);
    printf("float: %zu bytes, normal min=%.9g max=%.9g digits=%d
        mantissa=%d\n",sizeof(float),(double)FLT_MIN,(double)FLT_MAX,FLT_DIG,FLT_MANT_DIG);
    printf("double: %zu bytes, normal min=%.17g max=%.17g digits=%d
        mantissa=%d\n",sizeof(double),DBL_MIN,DBL_MAX,DBL_DIG,DBL_MANT_DIG);
    printf("long double: %zu bytes, max=%.9Lg mantissa=%d\n",sizeof(long
        double),LDBL_MAX,LDBL_MANT_DIG);
    uint16_t meter=UINT16_MAX;
    meter=(uint16_t)(meter+1U);
    printf("16-bit register after 65535 + 1: %u\n",(unsigned)meter);
    return 0;
}
```

Task03/complete_code.c

```
#include <stdio.h>
#define SLAB1_UNITS 100            /* first 100 units            */
#define SLAB2_UNITS 200            /* units 101 to 200           */
#define RATE1_PAISE 0              /* first slab is free         */
#define RATE2_PAISE 225            /* Rs 2.25 per unit           */
#define RATE3_PAISE 450            /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L   /* Rs 50 fixed charge         */
/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    if (units > SLAB2_UNITS) {
        paise += (long)(units-SLAB2_UNITS)*RATE3_PAISE;
        units=SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS) {
        paise += (long)(units-SLAB1_UNITS)*RATE2_PAISE;
        units=SLAB1_UNITS;
    }
    paise += (long)units*RATE1_PAISE;
    return paise;
}
int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Task03/revised_rate.c

```
#include <stdio.h>
#define SLAB1_UNITS 100            /* first 100 units            */
#define SLAB2_UNITS 200            /* units 101 to 200           */
#define RATE1_PAISE 0              /* first slab is free         */
#define RATE2_PAISE 230            /* Rs 2.25 per unit           */
#define RATE3_PAISE 450            /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L   /* Rs 50 fixed charge         */
/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    if (units > SLAB2_UNITS) {
        paise += (long)(units-SLAB2_UNITS)*RATE3_PAISE;
        units=SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS) {
        paise += (long)(units-SLAB1_UNITS)*RATE2_PAISE;
        units=SLAB1_UNITS;
    }
    paise += (long)units*RATE1_PAISE;
    return paise;
}
int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Task04/macros.c

```
#include <stdio.h>
#define BAD_SQUARE(x) x*x
#define SQUARE(x) ((x)*(x))
#define MAX(a,b) (((a)>(b))?(a):(b))
/* Whole non-negative rupees only; caller must check multiplication range. */
#define RUPEES_TO_PAISE(r) ((r)*100LL)
static inline int square_int(int x) { return x*x; }
static inline int max_int(int a,int b) { return a>b?a:b; }
int main(void) {
    int a=3, i=5, j=3;
    printf("BAD_SQUARE(a+1)=%d; SQUARE(a+1)=%d\n",BAD_SQUARE(a+1),SQUARE(a+1));
    int result=MAX(i++,j); /* Condition and selected arm are sequenced. */
    printf("MAX(i++,j)=%d; i after=%d\n",result,i);
    i=5;result=max_int(i++,j);
    printf("max_int(i++,j)=%d; i after=%d\n",result,i);
    printf("square_int(a+1)=%d; 50 rupees=%lld paise\n",square_int(a+1),RUPEES_TO_PAISE(50));
    return 0;
}
```

Task05/header_demo.c

```
#include <stdio.h>
#include "../common/billing.h"
#include "../common/billing.h" /* Repeated inclusion is harmless with guards. */
int main(void) {
    Bill b=bill_for_units(150);
    printf("Header guards work; total=");print_money(b.total_paise);putchar('\n');
    return 0;
}
```

Task06/scanf_demo.c

```
#include <stdio.h>
/* Teaching example: numeric token must be within int's range.
   Task09 uses bounded text plus checked conversion for arbitrary input. */
int main(void) {
    int previous=0,current=0;
    printf("Enter previous and current readings: ");
    int matched=scanf("%d %d",&previous,&current);
    printf("scanf assignments=%d\n",matched);
    if (matched!=2) { puts("Invalid numeric input");return 2; }
    if (previous<0 || current<0 || previous>65535 || current>65535) {
        puts("Reading outside 0..65535");return 2;
    }
    printf("Accepted previous=%d current=%d\n",previous,current);
    long p=16250L;
    printf("Example formatted bill: Rs %ld.%02ld\n",p/100,p%100);
    return 0;
}
```

Task06/wrong_format.c

```
/* Deliberately incorrect. Compile with -Werror=format; do not execute. */
#include <stdio.h>
int main(void) {
    long amount=2147483648L;
    printf("%d\n",amount);
    return 0;
}
```

Task07/conversions.c

```
#include <stdio.h>
#include <limits.h>
#include <math.h>
int main(void) {
    int units=150;
    printf("150/100=%d; (double)150/100=%.2f\n",units/100,(double)units/100);
    int negative=-1;unsigned one=1U;
    printf("-1 < 1U = %d (after unsigned conversion)\n",negative<one);
    printf("Signed-domain comparison corrected = %d\n",negative<0 || (unsigned)negative<one);
    unsigned char small=255;
    printf("unsigned char 255 + 1 = %d (integer promotion)\n",small+1);
    if (LONG_MAX>INT_MAX) {
        long large=(long)INT_MAX+1L;
        printf("(int)(INT_MAX+1L)=%d (implementation-defined in C17)\n",(int)large);
    }
    float f=16777216.0f;
    printf("float 16777216 + 1 = %.0f\n",(double)(f+1.0f));
    /* Non-negative charge; finite/range validation must precede llround. */
    double rupees=1.125;
    printf("llround(1.125*100)=%lld paise (half away from zero)\n",llround(rupees*100));
    return 0;
}
```

Task07/precision.c

```
#include <stdio.h>
#include <limits.h>
#include <time.h>
#include "../common/billing.h"
int main(void) {
    const unsigned sizes[]={4000,60000,800000};
    const money_t one=bill_for_units(65535).total_paise;
    puts("n,integer_paise,float_rupees,float_error_paise,double_rupees,double_error_paise,int_safe,
        long_safe,ll_safe,float_ms,integer_ms,double_ms");
    for (unsigned k=0;k<3;k++) {
        unsigned n=sizes[k];volatile float ft=0;volatile double dt=0;volatile money_t it=0;
        clock_t start=clock();
        for (unsigned i=0;i<n;i++) ft+=(float)one/100.0f;
        double ft_ms=1000.0*(double)(clock()-start)/CLOCKS_PER_SEC;
        start=clock();for (unsigned i=0;i<n;i++) it+=one;
        double it_ms=1000.0*(double)(clock()-start)/CLOCKS_PER_SEC;
        start=clock();for (unsigned i=0;i<n;i++) dt+=(double)one/100.0;
        double dt_ms=1000.0*(double)(clock()-start)/CLOCKS_PER_SEC;
        printf("%u,%lld,%.2f,%.0f,%.2f,%.0f,%d,%d,%d,%.6f,%.6f,%.6f\n",n,(money_t)it,(double)ft,(do
            uble)ft*100.0-(double)it,(double)dt,(double)dt*100.0-(double)it,
               one<=INT_MAX/n,one<=LONG_MAX/n,one<=LLONG_MAX/n,ft_ms,it_ms,dt_ms);
    }
    printf("First overflowing int addition at maximum bill: %lld\n",(money_t)INT_MAX/one+1);
    return 0;
}
```

Task08/compile_error.c

```
/* Deliberate compile-time error. */
int main(void) { return missing_variable; }
```

Task08/link_error.c

```
/* Declared, but no implementation is supplied for the link test. */
int missing_tariff(void);
int main(void) { return missing_tariff(); }
```

Task08/runtime_error.c

```
/* Defined run-time failure rather than undefined behaviour. */
#include <stdio.h>
int main(void) {
    FILE *f=fopen("__nonexistent_assignment_input__.txt","r");
    if (!f) { perror("Cannot open input");return 2; }
    fclose(f);return 0;
}
```

Task08/stages.c

```
#include <stdio.h>
#define CHARGE 5000L
int main(void) { printf("Fixed charge=%ld paise\n",CHARGE);return 0; }
```

Task09/input.c

```
#include "input.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <ctype.h>
static bool digits(const char *s) {
    if (!*s) return false;
    for (;*s;s++) if (*s<'0'||*s>'9') return false;
    return true;
}
static bool meter_value(const char *s,meter_t *out) {
    if (!digits(s)) return false;
    errno=0;char *end=NULL;
    unsigned long value=strtoul(s,&end,10);
    if (errno==ERANGE || *end || value>65535UL) return false;
    *out=(meter_t)value;return true;
}
bool parse_reading(const char *line,Reading *record) {
    char id[32],category[8],previous[32],current[32],roll[8],extra;
    if (sscanf(line,"%31s %7s %31s %31s %7s %c",id,category,previous,current,roll,&extra)!=5)
        return false;
    if (!digits(id) || strcmp(category,"D") || (strcmp(roll,"Y") && strcmp(roll,"N")))
        return false;
    if (!meter_value(previous,&record->previous)||!meter_value(current,&record->current))
        return false;
    strcpy(record->household,id);record->category='D';record->rollover=roll[0]=='Y';return true;
}
```

Task09/input.h

```
#ifndef CSA02_INPUT_H
#define CSA02_INPUT_H
#include "../common/billing.h"
typedef struct { char household[HOUSEHOLD_ID_SIZE];char category;
    meter_t previous,current;bool rollover; } Reading;
bool parse_reading(const char *line,Reading *record);
#endif
```

Task09/main.c

```
#include "input.h"
#include <stdio.h>
#include <string.h>
int main(void) {
    char line[256];unsigned line_number=0,accepted=0,rejected=0;
    money_t grand_total=0;
    while (fgets(line,sizeof line,stdin)) {
        ++line_number;
        if (!strchr(line,'\n') && !feof(stdin)) {
            int ch;while ((ch=getchar())!='\n'&&ch!=EOF) {}
            printf("REJECT line %u: line too long\n",line_number);++rejected;continue;
        }
        Reading r;unsigned units;
        if (!parse_reading(line,&r)) {
            printf("REJECT line %u: invalid
                fields/range/category\n",line_number);++rejected;continue;
        }
        if (!consumption_checked(r.previous,r.current,r.rollover,&units)) {
            printf("REJECT line %u: inconsistent or unconfirmed
                rollover\n",line_number);++rejected;continue;
        }
        Bill b=bill_for_units(units);
        if (!add_money(&grand_total,b.total_paise)) {
            fprintf(stderr,"Total overflow\n");return 3;
        }
        printf("%s units=%u energy=",r.household,units);print_money(b.energy_paise);
        printf(" fixed=");print_money(b.fixed_paise);printf("
            total=");print_money(b.total_paise);putchar('\n');
        ++accepted;
    }
    if (ferror(stdin)) { perror("Input error");return 3; }
    printf("Accepted=%u rejected=%u
        total=",accepted,rejected);print_money(grand_total);putchar('\n');
    return rejected?2:0;
}
```

Task09/tests.c

```
#include "input.h"
#include <assert.h>
#include <stdio.h>
#include <limits.h>
static money_t array_bill(unsigned units) {
    const unsigned widths[]={100,100,UINT_MAX};
    const money_t rates[]={0,225,450};money_t total=5000;
    for (unsigned i=0;i<3&&units;i++) {
        unsigned take=units<widths[i]?units:widths[i];
        total+=(money_t)take*rates[i];units-=take;
    }
    return total;
}
int main(void) {
    const unsigned u[]={0,99,100,101,150,199,200,201,350,65535};
    const money_t expected[]={5000,5000,5000,5225,16250,27275,27500,27950,95000,29428250};
    for (unsigned i=0;i<sizeof u/sizeof u[0];i++) {
        Bill b=bill_for_units(u[i]);assert(b.total_paise==expected[i]);
        assert(b.total_paise==b.energy_paise+b.fixed_paise);assert(b.energy_paise>=0);
        printf("units=%u expected=%lld actual=%lld PASS\n",u[i],expected[i],b.total_paise);
    }
    for (unsigned i=0;i<=65535;i++) assert(bill_for_units(i).total_paise==array_bill(i));
    unsigned consumed=0;
    assert(consumption_checked(65400,120,true,&consumed)&&consumed==256);
    assert(consumption_checked(65535,0,true,&consumed)&&consumed==1);
    assert(!consumption_checked(200,100,false,&consumed));
    assert(!consumption_checked(100,200,true,&consumed));
    Reading r;
    assert(parse_reading("0001 D 00000 00150 N",&r));
    const char *bad[]={"0001 D hello 150 N","0001 D -1 150 N","0001 D 0 65536 N",
        "0001 X 0 150 N","0001 D 0 150 N trailing","0001 D 99999999999999999999999 2 N"};
    for (unsigned i=0;i<sizeof bad/sizeof bad[0];i++) assert(!parse_reading(bad[i],&r));
    money_t total=0;const unsigned five[]={0,100,150,200,350};
    for (unsigned i=0;i<5;i++) assert(add_money(&total,bill_for_units(five[i]).total_paise));
    assert(total==148750);
    total=LLONG_MAX;assert(!add_money(&total,1));assert(total==LLONG_MAX);
    puts("65,536 array-vs-fixed comparisons; rollover, parser, sum and overflow guards: PASS");
    puts("Five hand-calculated bills total: 148750 paise = Rs 1487.50");
    return 0;
}
```

Task10/totals.c

```
#include <stdio.h>
#include <limits.h>
#include <time.h>
#include "../common/billing.h"
int main(void) {
    const unsigned sizes[]={4000,60000,800000};
    const money_t one=bill_for_units(65535).total_paise;
    puts("n,integer_paise,float_rupees,float_error_paise,double_rupees,double_error_paise,int_safe,
        long_safe,ll_safe,float_ms,integer_ms,double_ms");
    for (unsigned k=0;k<3;k++) {
        unsigned n=sizes[k];volatile float ft=0;volatile double dt=0;volatile money_t it=0;
        clock_t start=clock();
        for (unsigned i=0;i<n;i++) ft+=(float)one/100.0f;
        double ft_ms=1000.0*(double)(clock()-start)/CLOCKS_PER_SEC;
        start=clock();for (unsigned i=0;i<n;i++) it+=one;
        double it_ms=1000.0*(double)(clock()-start)/CLOCKS_PER_SEC;
        start=clock();for (unsigned i=0;i<n;i++) dt+=(double)one/100.0;
        double dt_ms=1000.0*(double)(clock()-start)/CLOCKS_PER_SEC;
        printf("%u,%lld,%.2f,%.0f,%.2f,%.0f,%d,%d,%d,%.6f,%.6f,%.6f\n",n,(money_t)it,(double)ft,(do
            uble)ft*100.0-(double)it,(double)dt,(double)dt*100.0-(double)it,
               one<=INT_MAX/n,one<=LONG_MAX/n,one<=LLONG_MAX/n,ft_ms,it_ms,dt_ms);
    }
    printf("First overflowing int addition at maximum bill: %lld\n",(money_t)INT_MAX/one+1);
    return 0;
}
```

Task11/recommendation.c

```
#include <stdio.h>
#include <limits.h>
#include "../common/billing.h"
int main(void) {
    money_t one=bill_for_units(65535).total_paise;
    money_t district=one*800000LL;
    printf("Maximum monthly bill=%lld paise\nDistrict bound=%lld paise\n",one,district);
    printf("long long maximum=%lld; bound fits=%s\n",LLONG_MAX,district<=LLONG_MAX?"yes":"no");
    puts("Use checked integer paise, a confirmed rollover flag, and one tariff definition.");
    return 0;
}
```

Task12/main.c

```
#include "smart.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <inttypes.h>
static bool value(const char *s,uint64_t *out) {
    if (!*s) return false;
    for (const char *p=s;*p;p++) if (*p<'0'||*p>'9') return false;
    errno=0;char *end=NULL;unsigned long long n=strtoull(s,&end,10);
    if (errno==ERANGE||*end||n>UINT64_MAX) return false;
    *out=(uint64_t)n;return true;
}
int main(int argc,char **argv) {
    if (argc!=2) { fprintf(stderr,"Usage: smart tariff.cfg < intervals.txt\n");return 2; }
    Tariff tariff;
    if (!load_tariff(argv[1],&tariff)) { fprintf(stderr,"Invalid tariff configuration\n");return 2;
         }
    Account account={0,0,0};char line[256];uint64_t last_key=0,last_reading=0;
    while (fgets(line,sizeof line,stdin)) {
        char ds[32],ss[32],ps[32],cs[32],extra;uint64_t day,slot,previous,current;
        if ((!strchr(line,'\n')&&!feof(stdin)) ||
            sscanf(line,"%31s %31s %31s %31s %c",ds,ss,ps,cs,&extra)!=4 ||
            !value(ds,&day)||!value(ss,&slot)||!value(ps,&previous)||!value(cs,&current)||
            day<1||day>31||slot>95||current<previous) {
            fprintf(stderr,"Invalid interval; billing aborted\n");return 2;
        }
        uint64_t key=(day-1)*96+slot;
        if (account.intervals&&(key!=last_key+1||previous!=last_reading)) {
            fprintf(stderr,"Missing, repeated or discontinuous interval\n");return 2;
        }
        if (!apply_interval(&account,&tariff,current-previous,(unsigned)slot)) {
            fprintf(stderr,"Interval limit/overflow rejected\n");return 2;
        }
        last_key=key;last_reading=current;
    }
    if (ferror(stdin)||!account.intervals) { fprintf(stderr,"No complete input\n");return 2; }
    uint64_t energy,total;
    if (!finish_bill(&account,&tariff,&energy,&total)) return 3;
    printf("Intervals=%" PRIu64 " used_Wh=%" PRIu64 "\n",account.intervals,account.used_wh);
    printf("Energy=%" PRIu64 " paise; fixed=%" PRIu64 " paise; total=Rs %" PRIu64 ".%02" PRIu64
        "\n",
           energy,tariff.fixed,total/100,total%100);
    puts("One supplied billing-period sequence; coverage must be checked before issuing a bill.");
    return 0;
}
```

Task12/smart.c

```
#include "smart.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>
static bool add(uint64_t x,uint64_t y,uint64_t *out) {
    if (x>UINT64_MAX-y) return false;
    *out=x+y;return true;
}
static bool mul(uint64_t x,uint64_t y,uint64_t *out) {
    if (y && x>UINT64_MAX/y) return false;
    *out=x*y;return true;
}
bool load_tariff(const char *path,Tariff *t) {
    const char *names[]={"slab1_wh","slab2_wh","rate1_paise","rate2_paise","rate3_paise",
        "fixed_paise","peak_start","peak_end","peak_percent","max_interval_wh"};
    uint64_t values[10]={0};unsigned seen=0;char line[160];
    FILE *f=fopen(path,"r");if (!f) return false;
    bool ok=true;
    while (fgets(line,sizeof line,f)) {
        char key[40],value[40],extra;
        if (!strchr(line,'\n')&&!feof(f)) { ok=false;break; }
        if (sscanf(line,"%39[^=]=%39s %c",key,value,&extra)!=2) { ok=false;break; }
        for (const char *p=value;*p;p++) if (*p<'0'||*p>'9') ok=false;
        if (!ok) break;
        unsigned i;for (i=0;i<10&&strcmp(key,names[i]);i++) {}
        if (i==10 || (seen&(1U<<i))) { ok=false;break; }
        errno=0;char *end=NULL;unsigned long long number=strtoull(value,&end,10);
        if (errno==ERANGE || *end || number>UINT64_MAX) { ok=false;break; }
        values[i]=(uint64_t)number;seen|=1U<<i;
    }
    if (ferror(f)) ok=false;
    fclose(f);
    if (!ok || seen!=1023U) return false;
    if (!values[0]||values[0]>=values[1]||values[1]>1000000000ULL ||
        values[2]>100000 || values[3]>100000 || values[4]>100000 ||
        values[5]>1000000 || values[6]>=values[7] || values[7]>24 ||
        values[8]>100 || !values[9] || values[9]>1000000) return false;
    *t=(Tariff){values[0],values[1],values[2],values[3],values[4],values[5],
                values[6],values[7],values[8],values[9]};
    return true;
}
bool apply_interval(Account *a,const Tariff *t,uint64_t delta_wh,unsigned slot) {
    if (!a || !t || slot>95 || delta_wh>t->max_interval_wh) return false;
    Account next=*a;uint64_t remaining=delta_wh;
    unsigned hour=slot/4;
    uint64_t factor=100+((hour>=t->peak_start&&hour<t->peak_end)?t->peak_percent:0);
    while (remaining) {
        uint64_t rate,capacity;
        if (next.used_wh<t->slab1_wh) { rate=t->rate1;capacity=t->slab1_wh-next.used_wh; }
        else if (next.used_wh<t->slab2_wh) { rate=t->rate2;capacity=t->slab2_wh-next.used_wh; }
        else { rate=t->rate3;capacity=remaining; }
        uint64_t take=remaining<capacity?remaining:capacity,term;
        if (!mul(take,rate,&term)||!mul(term,factor,&term)||
            !add(next.charge_numerator,term,&next.charge_numerator)||
            !add(next.used_wh,take,&next.used_wh)) return false;
        remaining-=take;
    }
    if (!add(next.intervals,1,&next.intervals)) return false;
    *a=next;return true; /* Commit only after every checked operation succeeds. */
}
bool finish_bill(const Account *a,const Tariff *t,uint64_t *energy,uint64_t *total) {
    uint64_t rounded=a->charge_numerator/100000;
    if (a->charge_numerator%100000>=50000 && !add(rounded,1,&rounded)) return false;
    if (!add(rounded,t->fixed,total)) return false;
    *energy=rounded;return true;
}
```

Task12/smart.h

```
#ifndef CSA02_SMART_H
#define CSA02_SMART_H
#include <stdbool.h>
#include <stdint.h>
typedef struct {
    uint64_t slab1_wh,slab2_wh,rate1,rate2,rate3,fixed;
    uint64_t peak_start,peak_end,peak_percent,max_interval_wh;
} Tariff;
typedef struct { uint64_t used_wh,charge_numerator,intervals; } Account;
/* numerator/100000 is the exact charge in paise before final rounding. */
bool load_tariff(const char *path,Tariff *t);
bool apply_interval(Account *a,const Tariff *t,uint64_t delta_wh,unsigned slot);
bool finish_bill(const Account *a,const Tariff *t,uint64_t *energy,uint64_t *total);
#endif
```

Task12/tests.c

```
#include "smart.h"
#include <assert.h>
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
int main(int argc,char **argv) {
    assert(argc==2);Tariff t;assert(load_tariff(argv[1],&t));
    Account a={0,0,0};uint64_t energy,total;
    assert(apply_interval(&a,&t,100000,71));
    assert(apply_interval(&a,&t,50000,72));
    assert(apply_interval(&a,&t,50000,73));
    assert(apply_interval(&a,&t,150000,74));
    assert(finish_bill(&a,&t,&energy,&total)&&energy==108000&&total==113000);
    printf("350 kWh mixed/slab-crossing test: energy=%" PRIu64 " total=%" PRIu64 " paise
        PASS\n",energy,total);
    const unsigned slots[]={71,72,87,88};const uint64_t expected[]={450,540,540,450};
    for (unsigned i=0;i<4;i++) {
        a=(Account){200000,0,0};assert(apply_interval(&a,&t,1000,slots[i]));
        assert(finish_bill(&a,&t,&energy,&total)&&energy==expected[i]);
        printf("slot=%u (start %02u:%02u) energy=%" PRIu64 " paise
            PASS\n",slots[i],slots[i]/4,slots[i]%4*15,energy);
    }
    a=(Account){200000,0,0};assert(apply_interval(&a,&t,1,0));assert(apply_interval(&a,&t,1,1));
    assert(finish_bill(&a,&t,&energy,&total)&&energy==1);
    puts("Two 0.45-paisa intervals -> 1 paisa after one final rounding: PASS");
    a=(Account){200000,UINT64_MAX-1,0};Account before=a;
    assert(!apply_interval(&a,&t,1000,0));assert(a.charge_numerator==before.charge_numerator&&a.use
        d_wh==before.used_wh);
    assert(!apply_interval(&a,&t,t.max_interval_wh+1,0));
    puts("Overflow and over-limit rejection leave account unchanged: PASS");
    a=(Account){0,0,0};
    for (unsigned day=0;day<365;day++) for (unsigned slot=0;slot<96;slot++)
        assert(apply_interval(&a,&t,1000,slot));
    assert(a.intervals==35040&&a.used_wh==35040000);
    printf("Annual interval traversal=%" PRIu64 " count; total=%" PRIu64 " Wh
        PASS\n",a.intervals,a.used_wh);
    return 0;
}
```

common/billing.c

```
#include "billing.h"
#include <stdio.h>
bool consumption_checked(meter_t previous, meter_t current,
                         bool rollover_confirmed, unsigned *units) {
    if (!units) return false;
    /* At most one wrap and strictly less than 65536 units per period. */
    if ((current < previous) != rollover_confirmed) return false;
    *units = (uint16_t)((unsigned)current - (unsigned)previous);
    return true;
}
Bill bill_for_units(unsigned units) {
    Bill b = {units, 0, FIXED_CHARGE_PAISE, 0};
    unsigned remaining = units;
    if (remaining > SLAB2_UNITS) {
        b.energy_paise += (money_t)(remaining - SLAB2_UNITS) * RATE3_PAISE;
        remaining = SLAB2_UNITS;
    }
    if (remaining > SLAB1_UNITS) {
        b.energy_paise += (money_t)(remaining - SLAB1_UNITS) * RATE2_PAISE;
        remaining = SLAB1_UNITS;
    }
    b.energy_paise += (money_t)remaining * RATE1_PAISE;
    b.total_paise = b.energy_paise + b.fixed_paise;
#ifdef DEBUG
    fprintf(stderr, "DEBUG units=%u energy=%lld fixed=%lld total=%lld\n",
            units,b.energy_paise,b.fixed_paise,b.total_paise);
#endif
    return b;
}
void print_money(money_t paise) {
    printf("Rs %lld.%02lld", paise/100, paise%100);
}
bool add_money(money_t *total, money_t amount) {
    if (!total || *total < 0 || amount < 0 || *total > LLONG_MAX-amount)
        return false;
    *total += amount;
    return true;
}
```

common/billing.h

```
#ifndef CSA02_BILLING_H
#define CSA02_BILLING_H
#include <stdint.h>
#include <stdbool.h>
#include <limits.h>
#define SLAB1_UNITS 100U
#define SLAB2_UNITS 200U
#define RATE1_PAISE 0LL
#ifndef RATE2_PAISE
#define RATE2_PAISE 225LL
#endif
#define RATE3_PAISE 450LL
#define FIXED_CHARGE_PAISE 5000LL
#define METER_MODULUS 65536UL
#define HOUSEHOLD_ID_SIZE 32
_Static_assert(UINT16_MAX == 65535, "Requires a 16-bit unsigned type");
typedef uint16_t meter_t;
typedef long long money_t;
typedef struct {
    unsigned units;
    money_t energy_paise, fixed_paise, total_paise;
} Bill;
bool consumption_checked(meter_t previous, meter_t current,
                         bool rollover_confirmed, unsigned *units);
Bill bill_for_units(unsigned units);
void print_money(money_t paise);
bool add_money(money_t *total, money_t amount);
#endif
```